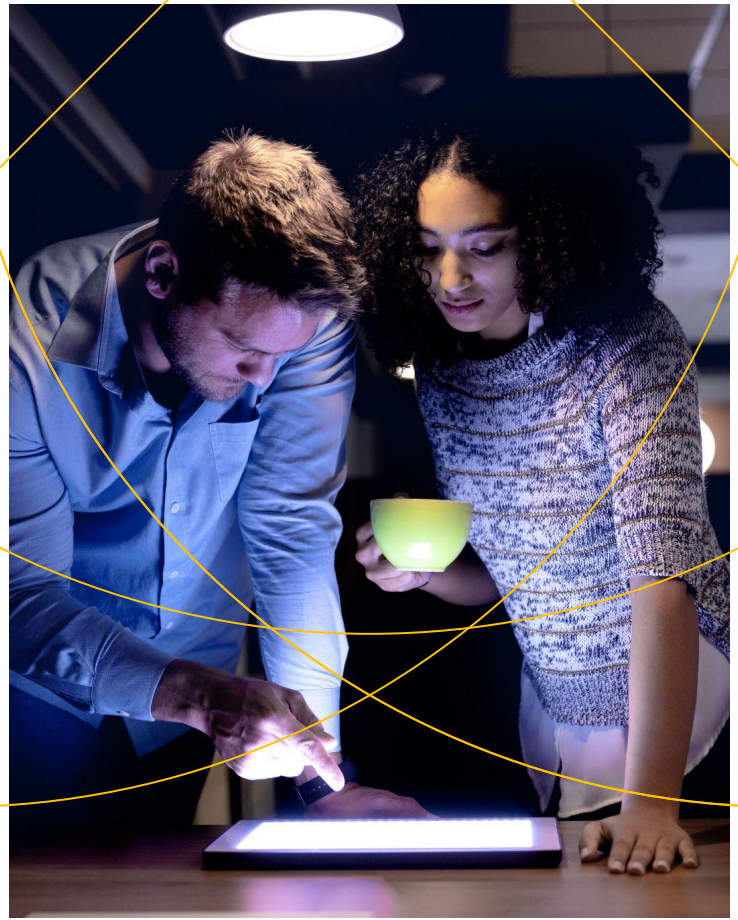
ippon

# From Monolith to AI Agent: Modernizing Java Systems with MCP



## AGENDA

### PART 01

Intro

Legacy Java monoliths in the AI era

### PART 02

Introducing MCP

Steps to refactor a Java monolith

### PART 03

Key patterns

Takeaways and Q&A

# INTRODUCTION
## Théo Lebrun

lebruntheo

Falydoor

@Falydoor

https://theolebrun.com/

Data Engineer and SME at Ippon Technologies

# Legacy Java Systems Today

- **Power critical enterprise workflows**
  - Remain isolated silos
  - Java is one (or the one) top choice for enterprise

- **Struggle to integrate modern AI/LLM capabilities**
  - Expensive
  - Risky and often requires full rewrites
  - Spring AI requires Java 17+

- **Missed opportunities**
  - Missing out on AI-driven automation & intelligence
  - Leaving teams stuck

# Model Context Protocol (MCP)

- **Open standard**
  - Introduced by Anthropic on Nov. 2024
  - Now widely adopted

- **Client-server model**
  - AI (host) <> your systems (MCP servers)
  - “APIs” for AI

- **USB-C for AI integrations**
  - Vendor-neutral & secure
  - External data sources, tools, and databases

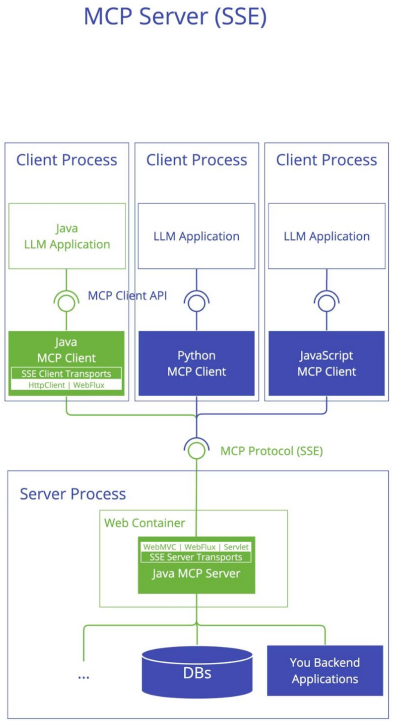
# Why MCP for Java Developers?

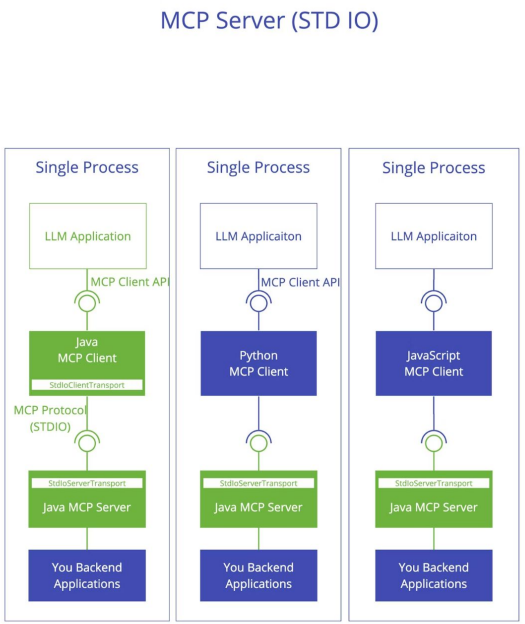
- **No need to rewrite monoliths**
  - Keep battle-tested Java code
  - Expose existing services as first-class tools to LLMs

- **Java stays relevant in fast-moving AI ecosystem**
  - Without sacrificing reliability

- **MCP Frameworks**
  - Spring AI
  - MCP Java SDK
  - LangChain4j

# MCP Architecture Overview




https://modelcontextprotocol.io/sdk/java/mcp-overview

# Case Study – Legacy Java Monolith

- **Typical enterprise app**
  - REST APIs
  - Business logic
  - Databases
  - No native AI integration

- **Goal**
  - Transform into intelligent agent without disruption

- **Example domain**
  - Order processing
  - Customer support
  - Inventory management

# Step 1 – Exposing Services as MCP Tools

- **Identify key business functions**
  - queryOrder
  - updateStatus
  - getOrder
  - etc.
- **Annotate/Implement**
  - Spring AI
  - MCP Java SDK
- **Define tool schemas**
  - Inputs/outputs for LLM discovery
  - Minimal changes to existing code base

# Step 2 – Handling Tool Calls

- **Handled by your LLM**
  - When to call
  - Which tools

- **Flow**
  - MCP routes call
  - Server executes
  - Returns result

- **Built-in features**
  - Error handling
  - Retries system

# Step 3 – Streaming Results

- **Support streaming responses**
  - Long-running operations
  - Better UX
  - Improve “perceived” performance

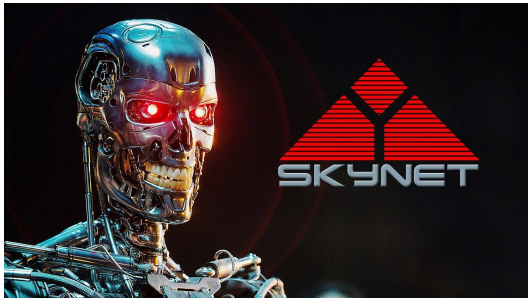
- **Agent reasoning**
  - Progressive/iterative reasoning
  - Reduces context window pressure and hallucination risk
  - Error recovery and self-correction

- **Provide progressive updates to LLM/user**
  - Streaming query results
  - Live status updates

# Production Strategies – Control & Audit

- **Compliance & MCP server**
  - Authentication & authorization
  - Log & audit every tool call/action

- **Sensitive operations**
  - Human-in-the-loop
  - Rate limiting
  - Circuit breakers



# Benefits & Results

- **Faster AI adoption**
  - Without sacrificing Java stability

- **Composable agents**
  - Combine multiple MCP servers
  - Works with evolving LLMs & frameworks

- **Reduced integration cost & technical debt**
  - Standardized connectors and no rewrites
  - No “glue” code

# Key Takeaways

- **MCP bridges legacy Java systems to modern AI agents**
  - Start small
  - One service at a time to full agent transformation

- **Practical path**
  - Expose => Handle calls => Stream => Control

- **For quick wins**
  - Spring AI
  - MCP Java SDK

# Useful Links

- **https://spring.io/projects/spring-ai**
  - Connecting your enterprise Data and APIs with AI Models

- **https://github.com/langchain4j/langchain4j**
  - LLM Framework for Java

- **https://modelcontextprotocol.io/sdk/java/mcp-overview**
  - “Official” SDK

- **https://github.com/jdubois/dr-jskill**
  - Agent Skill for creating Spring Boot applications
  - Work with tools like GitHub Copilot CLI or Claude Code

# THANK YOU

**You can email me at tlebrun@ipponusa.com or message me on in**

# Q & A